

Hocus Pocus, Spanish Pocus

(I cannot have a cover, word does not exist on Linux and I can't add a cover on the web version)

Table of Contents

<i>Introduction: Hocus Pocus, Spanish Pokus.....</i>	2
Authors.....	3
Problem Statement.....	3
Solution	3
Implementation	3
<i>API Specification</i>	3
Authentication	5
Errors.....	5
Endpoints	6
Resource 1 (e.g. users).....	7

Introduction: Hocus Pocus, Spanish Pokus

Authors

- Joakim Vatnehol Pedersen

Problem Statement

Design and implement a RESTful API for a Spanish Poker Dice platform that supports multiple game categories defined by combinations of round counts, rule variations, and time controls. The API should enable gameplay between users on separate machines, unlike previous single-device implementations, while focusing only on backend support (excluding real-time gameplay features such as WebSocket communication) and the front end part is covered by a colleague. Additionally, the system must account for different user types like anonymous users, registered users, and administrators—through simplified authentication and authorization mechanisms.

Solution

Our suggested backend solution is based on a client-server. The backend exposes HTTP endpoints that allow a separate frontend to do things like:

- Manage users (anonymous allowed)
- Create, join and complete matches for the Spanish Poker game.
- Organize and participate in tournaments,
- View the leaderboard based on wins and win rate.
- See and inspect the ongoing matches and active users

Implementation

This got implemented by using:

- NodeJS and Express for HTTP server and routing.
- MongoDB and Mongoose for the database and persistence (schema)
- The implementation is also layered with:
 - o Routes: defined URL structure and HTTP
 - o Controllers: handle requests and response mappings.
 - o Services: contains the business logic.
 - o Models: Defines the schemas we use for users, matches and tournaments.
 - o Validation: There is also a validation layer, it is important it validates on both points (before and after DB)

API Specification

Authentication

The current implementation uses simplified authentication:

- Anonymous users can be presented without full registration
- Registered users are stored in database with hashed passwords
- Admin like behaviour can be simulated via flags on user document

Errors

The API uses standard HTTP status codes and JSON error responses.

HTTP status code	Error code	Description
400	BAD_REQUEST	Invalid or missing input data, validation errors, or malformed request.
401	UNAUTHORIZED	Missing or invalid authentication details (conceptual, if auth is enforced).
403	FORBIDDEN	User is authenticated but not allowed to perform this action.
404	NOT_FOUND	Resource not found (user, match, tournament, etc.).
409	CONFLICT	Conflicting state (e.g., joining a full match or tournament).
429	TOO_MANY_REQS	Too many requests in a short time (if rate limiting is added).
500	SERVER_ERROR	Unexpected server error or unhandled exception.

Endpoints

Resource 1: Users

URI (verb and route)	Description	Inputs	Outputs
GET /api/users	list all users	query params for filtering (optional)	array of user objects
GET /api/users/:uid	get a single user by id	route param uid	user object
GET /api/users/:uid/profile	get profile info for a user	route param uid	profile object (subset/extended user data)
POST /api/users	create a new user	JSON body with user fields (e.g., username, email, password, etc.)	created user object

Resource 2: Matches

URI (verb + route)	Description	Inputs	Outputs (200/201)
POST /api/matches	create a new match	JSON body with match settings (rounds, rules, time control, players)	created match object
POST /api/matches/:id/join	join an existing match	route param id, body with joining user info	updated match object
POST /api/matches/:id/result	save match result	route param id, body with result data (winner, scores, etc.)	updated match object
POST /api/matches/:id/comments	add a comment to a match	route param id, body with comment text and author	updated match object with new comment

Resource 3: Tournaments

URI (verb + route)	Description	Inputs	Outputs (200/201)
POST /api/tournaments	create a new tournament	JSON body with tournament settings (name, rounds, rules, etc.)	created tournament object
POST /api/tournaments/:id/join	join a tournament	route param id, body with user info	updated tournament object

POST /api/tournaments/:id/advance	advance players/rounds in a tournament	route param id, body with advancement data	updated tournament object
POST /api/tournaments/:id/comments	add a comment to a tournament	route param id, body with comment text and author	updated tournament object

Resource 4: Leaderboards

URI (verb + route)	Description	Inputs	Outputs (200)
GET /api/leaderboards/wins	leaderboard sorted by most wins	none	array of users with wins field
GET /api/leaderboards/winrate	leaderboard sorted by best winrate	none	array of users with winrate field

Resource 5: Platform

URI (verb + route)	Description	Inputs	Outputs (200)
GET /api/platform	platform activity overview	none	object with ongoing games, active users, recent games